Cheat Sheet: Linear and Logistic Regression

1 Comparing different regression types

Model Name	Description	Code Syntax
Simple linear regression	**Purpose:** To predict a dependent variable based on one independent variable. **Pros:** Easy to implement, interpret, and efficient for small datasets. **Cons:** Not suitable for complex relationships; prone to underfitting. **Modeling equation:** $y = b_0 + b_1x$	1. `from sklearn.linear_model import LinearRegression` 2. `model = LinearRegression()` 3. `model.fit(X, y)`
Polynomial regression	**Purpose:** To capture nonlinear relationships between variables. **Pros:** Better at fitting nonlinear data compared to linear regression. **Cons:** Prone to overfitting with high-degree polynomials. **Modeling equation:** $y = b_0 + b_1x + b_2x^2 + ...$	1. `from sklearn.preprocessing import PolynomialFeatures` 2. `from sklearn.linear_model import LinearRegression` 3. `poly = PolynomialFeatures(degree=2)` 4. `X_poly = poly.fit_transform(X)` 5. `model = LinearRegression().fit(X_poly, y)`
Multiple linear regression	**Purpose:** To predict a dependent variable based on multiple independent variables. **Pros:** Accounts for multiple factors influencing the outcome.	1. `from sklearn.linear_model import LinearRegression` 2. `model = LinearRegression()` 3. `model.fit(X, y)`

Model Name	Description	Code Syntax
	Cons: Assumes a linear relationship between predictors and target. **Modeling equation:** $y = b_0 + b_1x_1 + b_2x_2 + ...$	
Logistic regression	**Purpose:** To predict probabilities of categorical outcomes. **Pros:** Efficient for binary classification problems. **Cons:** Assumes a linear relationship between independent variables and log-odds. **Modeling equation:** $\log(p/(1-p)) = b_0 + b_1x_1 + ...$	1. `from sklearn.linear_model import LogisticRegression` 2. `model = LogisticRegression()` 3. `model.fit(X, y)`

2 Associated functions commonly used

Function/Method Name	Brief Description	Code Syntax
train_test_split	Splits the dataset into training and testing subsets to evaluate the model's performance.	1. `from sklearn.model_selection import train_test_split` 2. `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`
StandardScaler	Standardizes features by removing the mean and	1. `from sklearn.preprocessing import StandardScaler` 2. `scaler = StandardScaler()` 3. `X_scaled = scaler.fit_transform(X)`

Function/Method Name	Brief Description	Code Syntax
	scaling to unit variance.	
log_loss	Calculates the logarithmic loss, a performance metric for classification models.	1. `from sklearn.metrics import log_loss` 2. `loss = log_loss(y_true, y_pred_proba)`
mean_absolute_error	Calculates the mean absolute error between actual and predicted values.	1. `from sklearn.metrics import mean_absolute_error` 2. `mae = mean_absolute_error(y_true, y_pred)`
mean_squared_error	Computes the mean squared error between actual and predicted values.	1. `from sklearn.metrics import mean_squared_error` 2. `mse = mean_squared_error(y_true, y_pred)`
root_mean_squared_error	Calculates the root mean squared error (RMSE), a commonly used metric for regression tasks.	1. `from sklearn.metrics import mean_squared_error` 2. `import numpy as np` 3. `rmse = np.sqrt(mean_squared_error(y_true, y_pred))`
r2_score	Computes the R-squared value, indicating how well the model explains the variability of the target variable.	1. `from sklearn.metrics import r2_score` 2. `r2 = r2_score(y_true, y_pred)`